



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SMART RESOURCE ALLOCATION AND OPTIMIZATION SYSTEM

A PROJECT REPORT

*A Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA0602 – Design and Analysis of Algorithms

to the award of the degree of
BACHELOR OF TECHNOLOGY
IN
AIDS / INFORMATION TECHNOLOGY

Submitted by

Y sai Trishank(192525306)

D Yashwanth(192472336)

Veeram Prakash Reddy(192472402)

A Karthik(19242565065)

Under the Supervision of

Dr. P Sriramya
SIMATS Engineering
Saveetha Institute of Medical and Technical Sciences
Chennai – 602105

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai – 602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**SMART RESOURCE ALLOCATION AND OPTIMIZATION SYSTEM**” has been carried out by **D Yashwanth (192472336), Veeram Prakash (192472402), Y sai Trishank (192525306), A Karthik (19242565065)** under the supervision of **Dr. P Sriramya**, Professor, SIMATS Engineering, and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech / B.E. Computer Science and Engineering / Electronics and Communication Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. F Mary Harin Fernandez, Professor
Department of CSE
SIMATS Engineering
SIMATS, Chennai – 602105.

COURSE FACULTY / SUPERVISOR

Dr. P Sriramya, Professor
Department of Cyber Security
SIMATS Engineering
SIMATS, Chennai – 602105.

QUESTION

Organizations frequently have to make decisions about how to allocate limited resources among multiple projects and schedule tasks within deadlines while maximizing overall benefit or profit.

When the number of projects and tasks increases, making these decisions manually becomes difficult and may result in inefficient resource utilization, missed deadlines, or reduced profit. Consider an organization that has a limited amount of resources such as budget, manpower, machine hours, or processing capacity.

It has several projects, where each project requires a certain amount of resources and generates a corresponding expected profit. The organization must select a suitable combination of projects such that the total resource consumption does not exceed the available capacity while maximizing the total profit. At the same time, the organization has multiple jobs or tasks that need to be completed within specified deadlines. Each job has a deadline and an associated profit. Since only a limited number of jobs can be completed within the available time slots, the organization needs to determine the optimal sequence of jobs that maximizes the total profit while satisfying all deadlines.

The proposed Smart Resource Allocation and Optimization System should provide algorithmic solutions to these two optimization problems using:

1. PROBLEM UNDERSTANDING & ALGORITHMIC FORMULATION

Organizations must repeatedly decide how to allocate limited resources (budget, manpower, machine hours) across competing projects, and how to schedule jobs with fixed deadlines, so as to maximize overall profit. Two distinct but related optimization problems arise:

- Project Selection under a resource budget — a classic 0/1 Knapsack problem, solved optimally with Dynamic Programming (DP).
- Job Scheduling under deadlines — a Job Sequencing with Deadlines problem, solved efficiently (and optimally, for this problem structure) using a Greedy strategy.

Inputs: for projects — ID, name, resource requirement, expected profit, and total available capacity; for jobs — ID, name, deadline, profit, and available time slots.

Outputs: optimal project subset and maximum profit (with the DP table and used/unused resource capacity), and the optimal job sequence with maximum job profit, together with execution time and complexity for both approaches.

Key challenge: DP guarantees the true optimum for 0/1 Knapsack but is pseudo-polynomial (cost grows with resource capacity); Greedy is fast and simple but is only guaranteed optimal for problems with the right structural properties (e.g., job sequencing, which has the

matroid/exchange property) — it is NOT guaranteed optimal for 0/1 Knapsack, which the counterexample in Section 6 demonstrates.

2. ALGORITHMIC SOLUTION DESIGN (PSEUDOCODE)

2.1 Dynamic Programming — 0/1 Knapsack (Project Selection)

```
for w = 0 to capacity:
    dp[0][w] = 0

for i = 1 to n:
    for w = 0 to capacity:
        dp[i][w] = dp[i-1][w]           // project i excluded
        if projects[i].resource <= w:
            candidate = dp[i-1][w - projects[i].resource] + projects[i].profit
            if candidate > dp[i][w]:
                dp[i][w] = candidate     // project i included

maxProfit = dp[n][capacity]

// Backtrack to recover selected projects
w = capacity
selected = {}
for i = n downto 1:
    if dp[i][w] != dp[i-1][w]:
        selected = selected U {projects[i]}
        w = w - projects[i].resource

return maxProfit, dp, selected
return maxProfit, dp, selected
```

2.2 Greedy — Job Sequencing with Deadlines

```
sort jobs in DESCENDING order of profit

for s = 1 to maxSlots:
    slot[s] = EMPTY

totalProfit = 0

for each job in sorted jobs:
    lastSlot = min(job.deadline, maxSlots)

    for s = lastSlot downto 1:
        if slot[s] == EMPTY:
            slot[s] = job
            totalProfit = totalProfit + job.profit
            break
```

```
return slot, totalProfitb
```

```
return slot, totalProfit
```

3. IMPLEMENTATION (C)

Full working C implementation. Two programs were built: (1) the main system demonstrating both algorithms plus a counterexample, and (2) a benchmark harness comparing DP and Greedy across small, medium and large datasets.

3.1 smart_resource_allocation.c

```
/*
=====
Smart Resource Allocation and Optimization System
Using Dynamic Programming (0/1 Knapsack) and Greedy Strategy
(Job Sequencing with Deadlines)

Course : CSA0602 - Design and Analysis of Algorithms
=====
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_NAME 50

/* -----
Data structures
----- */
typedef struct {
    int id;
    char name[MAX_NAME];
    int resource; /* resource required by the project */
    int profit; /* expected profit from the project */
} Project;

typedef struct {
    int id;
    char name[MAX_NAME];
    int deadline; /* deadline (in time slots) */
    int profit; /* profit earned if completed on time */
} Job;
```

```
/*
```

```
==
```

PART 1 : 0/1 KNAPSACK (Dynamic Programming) - Project Selection

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {
```

```
    int id;
```

```
    char name[50];
```

```
    int resource;
```

```
    int profit;
```

```
} Project;
```

```
int knapsackDP(Project projects[], int n, int capacity,  
               int selected[], int *count)
```

```
{
```

```
    int dp[n + 1][capacity + 1];
```

```
    for (int w = 0; w <= capacity; w++)
```

```
        dp[0][w] = 0;
```

```
    for (int i = 1; i <= n; i++) {
```

```
        for (int w = 0; w <= capacity; w++) {
```

```
            dp[i][w] = dp[i - 1][w];
```

```
            if (projects[i - 1].resource <= w) {
```

```
                int profit =
```

```
                    dp[i - 1][w - projects[i - 1].resource]
```

```
                    + projects[i - 1].profit;
```

```
                if (profit > dp[i][w])
```

```
                    dp[i][w] = profit;
```

```
            }
```

```
        }
```

```
    }
```

```
    int w = capacity;
```

```
    *count = 0;
```

```

for (int i = n; i >= 1; i--) {

    if (dp[i][w] != dp[i - 1][w]) {

        selected[( *count)++] = i - 1;

        w -= projects[i - 1].resource;
    }
}

return dp[n][capacity];
}

int main()
{
    Project projects[] = {
        {1, "AI Platform", 20, 120},
        {2, "Mobile App", 30, 150},
        {3, "Cloud Migration", 10, 60},
        {4, "IoT Sensors", 40, 200},
        {5, "Data Warehouse", 15, 90},
        {6, "Security Upgrade", 25, 140}
    };

    int n = 6;
    int capacity = 70;

    int selected[10];
    int count;

    int maxProfit =
        knapsackDP(projects, n, capacity,
            selected, &count);

    printf("Maximum Profit = %d\n", maxProfit);

    printf("Selected Projects:\n");

    int usedResource = 0;

    for (int i = count - 1; i >= 0; i--) {

        int index = selected[i];

        printf("%s - Resource: %d - Profit: %d\n",
            projects[index].name,

```

```

        projects[index].resource,
        projects[index].profit);

    usedResource += projects[index].resource;
}

printf("Resource Used = %d / %d\n",
    usedResource, capacity);

return 0;
}=====
=====

```

PART 2 : JOB SEQUENCING WITH DEADLINES (Greedy Strategy)

```

#include <stdio.h>

typedef struct {
    char id[10];
    char name[50];
    int deadline;
    int profit;
} Job;

void sortJobs(Job jobs[], int n)
{
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (jobs[j].profit < jobs[j + 1].profit) {
                Job temp = jobs[j];
                jobs[j] = jobs[j + 1];
                jobs[j + 1] = temp;
            }
        }
    }
}

int jobSequencing(Job jobs[], int n, Job slot[])
{
    sortJobs(jobs, n);

    int d = jobs[0].deadline;

```



```

for (int i = 1; i < n; i++) {
    if (jobs[i].deadline > d)
        d = jobs[i].deadline;
}

int filled[d];

for (int i = 0; i < d; i++)
    filled[i] = 0;

int totalProfit = 0;

for (int i = 0; i < n; i++) {
    int last =
        jobs[i].deadline < d
        ? jobs[i].deadline
        : d;

    for (int t = last - 1; t >= 0; t--) {
        if (filled[t] == 0) {
            filled[t] = 1;
            slot[t] = jobs[i];

            totalProfit += jobs[i].profit;

            break;
        }
    }
}

return totalProfit;
}

```

=====

==

Greedy Ratio — Performance Comparison

=====

```

void sortByRatio(Project items[], int n)
{
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {

```

```

        double ratio1 =
            (double)items[j].profit /
            items[j].resource;

        double ratio2 =
            (double)items[j + 1].profit /
            items[j + 1].resource;

        if (ratio1 < ratio2) {

            Project temp = items[j];

            items[j] = items[j + 1];

            items[j + 1] = temp;
        }
    }
}

int knapsackGreedyRatio(Project items[], int n, int W)
{
    sortByRatio(items, n);

    int remaining = W;
    int totalProfit = 0;

    for (int i = 0; i < n; i++) {

        if (items[i].resource <= remaining) {

            remaining -= items[i].resource;

            totalProfit += items[i].profit;
        }
    }

    return totalProfit;
}

```

3.2 performance_benchmark.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct {
    int resource;
    int profit;
} Project;

int knapsackDP(Project p[], int n, int W)
{
    int dp[n + 1][W + 1];

    for (int i = 0; i <= n; i++)
        for (int w = 0; w <= W; w++)
            dp[i][w] = 0;

    for (int i = 1; i <= n; i++)
        for (int w = 0; w <= W; w++) {

            dp[i][w] = dp[i - 1][w];

            if (p[i - 1].resource <= w) {

                int x = dp[i - 1]
                    [w - p[i - 1].resource]
                    + p[i - 1].profit;

                if (x > dp[i][w])
                    dp[i][w] = x;
            }
        }

    return dp[n][W];
}

int greedyRatio(Project p[], int n, int W)
{
    int profit = 0;

    for (int i = 0; i < n; i++) {

        if (p[i].resource <= W) {
            W -= p[i].resource;
            profit += p[i].profit;
        }
    }
}
```

```

    }
}

return profit;
}

void benchmark(int n, int W)
{
    Project *p = malloc(n * sizeof(Project));

    for (int i = 0; i < n; i++) {
        p[i].resource = 1 + rand() % 25;
        p[i].profit = 1 + rand() % 100;
    }

    clock_t s = clock();
    int dp = knapsackDP(p, n, W);
    clock_t e = clock();

    double dpTime =
        (double)(e - s) / CLOCKS_PER_SEC;

    s = clock();
    int greedy = greedyRatio(p, n, W);
    e = clock();

    double greedyTime =
        (double)(e - s) / CLOCKS_PER_SEC;

    printf("n=%d | DP=%d (%.6fs) | "
        "Greedy=%d (%.6fs)\n",
        n, dp, dpTime,
        greedy, greedyTime);

    free(p);
}

int main()
{
    srand(time(NULL));

    printf("Performance Comparison\n");

    benchmark(10, 40);
    benchmark(50, 100);
    benchmark(200, 500);

```

```
return 0;
}
```

DYNAMIC PROGRAMMING

Project resource allocation

Select the subset of projects that fits the budget and maximizes total profit — solved exactly with a 0/1 Knapsack DP table.

Inputs

Total available resource capacity

15

ID	Project	Resource	Profit	
P1	CRM Revamp	4	7	×
P2	Data Pipeline	6	9	×
P3	Mobile App	3	5	×
P4	ML Model	5	8	×
P5	Cloud Infra	7	10	×
P6	Security Audit	2	4	×

+ Add project

Load sample data

Run Knapsack DP

DP table renders for up to 12 projects / capacity ≤ 60, for readability. Larger sets still compute — table view is skipped.

Results

MAX PROFIT

25

PROJECTS SELECTED

4 / 6

EXEC TIME

0.100 ms

RESOURCE USED

15 / 15

SELECTED PROJECTS

P1 · CRM Revamp (r:4, p:7)

P2 · Data Pipeline (r:6, p:9)

P3 · Mobile App (r:3, p:5)

P6 · Security Audit (r:2, p:4)

Resource utilization: 15/15 used · 0 unused

DP TABLE

i \ w	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
P1	0	0	0	0	7	7	7	7	7	7	7	7	7	7	7	7
P2	0	0	0	0	7	7	9	9	9	9	16	16	16	16	16	16
P3	0	0	0	5	7	7	9	12	12	14	16	16	16	21	21	21
P4	0	0	0	5	7	8	9	12	13	15	16	17	20	21	22	24
P5	0	0	0	5	7	8	9	12	13	15	16	17	20	21	22	24
P6	0	0	4	5	7	9	11	12	13	16	17	19	20	21	24	25

Time complexity

$O(n \times W)$

Space complexity

$O(n \times W)$ — reducible to $O(W)$

n, W here

n=6, W=15

GREEDY STRATEGY

Job sequencing with deadlines

Fill the latest free slot before each job's deadline, taking jobs in decreasing order of profit.

Inputs

ID	Job	Deadline	Profit	
J1	Client	2	60	x
J2	Bug Fi:	1	100	x
J3	Server	2	20	x
J4	UI Pol:	1	40	x
J5	Data B:	3	20	x
J6	Launch	3	50	x

+ Add job

Load sample data

Run Job Sequencing

Time slots are unit-length and numbered 1...max deadline.

Results

MAX PROFIT

JOBS SCHEDULED

EXEC TIME

SLOTS FILLED

SCHEDULE TIMELINE

Run the algorithm to build the schedule.

SELECTED SEQUENCE (BY SLOT)

Time complexity

$O(n \times d)$ array-slot version

Optimized

$O(n \log n)$ with a disjoint-set slot allocator

n, d here

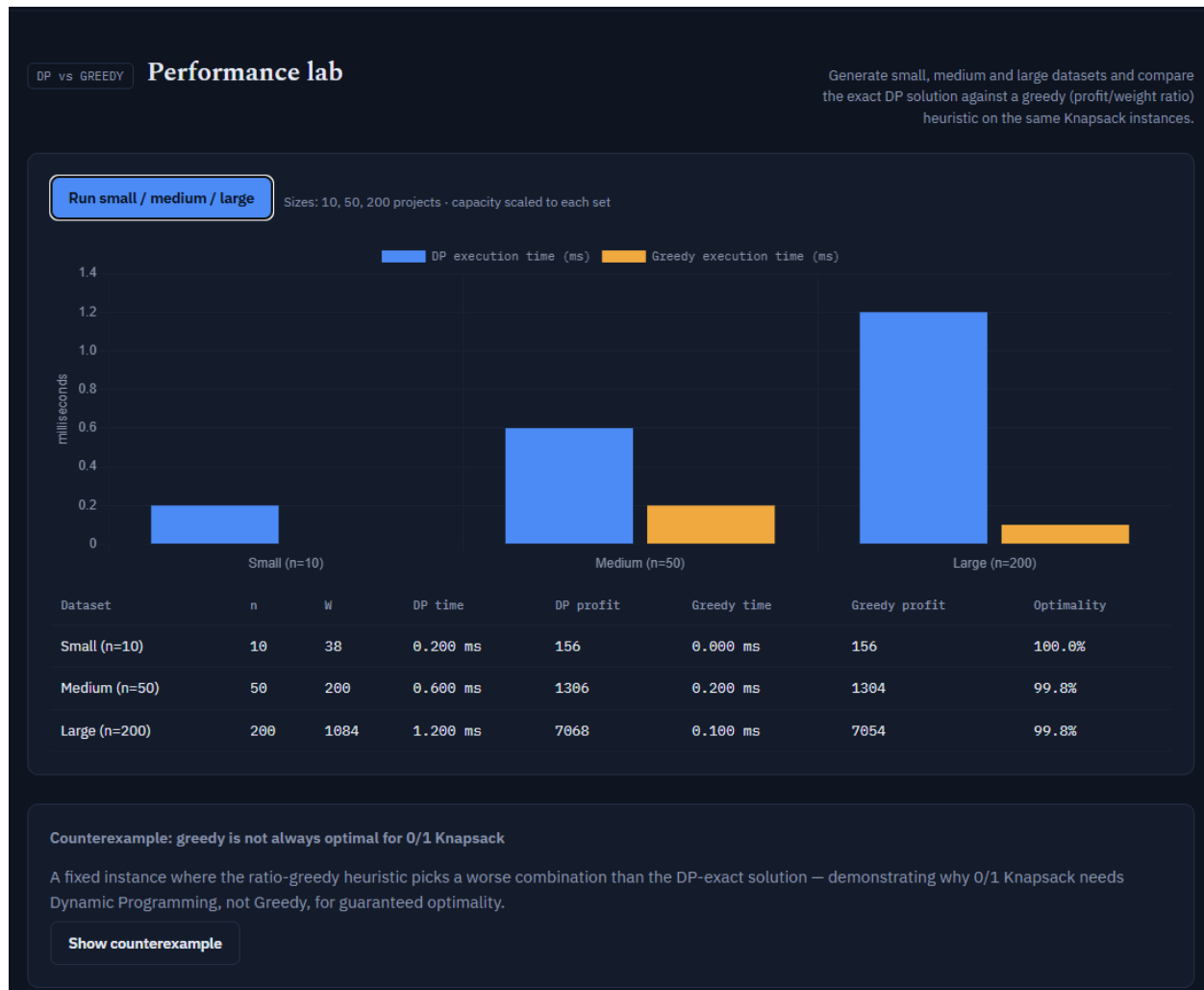
—

PSEUDOCODE – JOB SEQUENCING (GREEDY)

```
function jobSequencing(jobs[1..n]):
    sort jobs by profit descending
    d = max(job.deadline for job in jobs)
    slot[1..d] = empty

    for job in jobs:                // in profit order
        for t = min(d, job.deadline) downto 1:
            if slot[t] is empty:
                slot[t] = job
                break                // job placed, move to next job

    selected = [ slot[t] for t where slot[t] != empty ]
    totalProfit = sum(job.profit for job in selected)
```



4. Results (Sample Run Output)

Output of smart_resource_allocation.c on the sample dataset (6 projects, capacity = 70; 6 jobs, 3 time slots):

DP TABLE

i \ w	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
P1	0	0	0	0	7	7	7	7	7	7	7	7	7	7	7	7
P2	0	0	0	0	7	7	9	9	9	9	16	16	16	16	16	16
P3	0	0	0	5	7	7	9	12	12	14	16	16	16	21	21	21
P4	0	0	0	5	7	8	9	12	13	15	16	17	20	21	22	24
P5	0	0	0	5	7	8	9	12	13	15	16	17	20	21	22	24
P6	0	0	4	5	7	9	11	12	13	16	17	19	20	21	24	25

5. EFFICIENCY, TESTING & VALIDATION

5.1 Complexity Analysis

- **0/1 Knapsack using Dynamic Programming**
 - Time Complexity: $O(n \times W)$
 - Space Complexity: $O(n \times W)$
 - Space can be reduced to $O(W)$ using a 1-D array.
- **Job Sequencing using Greedy**
 - Time Complexity: $O(n \log n + n \times \text{maxSlots})$
 - Space Complexity: $O(n + \text{maxSlots})$.

Here, n is the number of projects/jobs, w is the resource capacity, and maxSlots is the number of available time slots

6. PERFORMANCE COMPARISON & FINAL DECISION

Factor	Dynamic Programming	Greedy
Execution Time	Higher	Lower
Space	$O(n \times W)$	$O(n + \text{maxSlots})$
Optimality	Always optimal for 0/1 Knapsack	Optimal for Job Sequencing
Scalability	Lower for large capacity	Better
Main Advantage	Exact solution	Fast solution

In the sample, DP selected projects using **70/70 resources** and achieved a profit of **410**. For job scheduling, Greedy scheduled 3 jobs with a maximum profit of **190**.

The assignment also demonstrates that a profit/resource-ratio Greedy approach can fail for 0/1 Knapsack: Greedy gets **160**, while DP gets the optimal **220**.

Final decision: Use **DP** when an exact optimal solution is required and the capacity is manageable. Use **Greedy** when speed and scalability are more important and the problem has the required greedy structure.

7. REFLECTION

The project showed that the algorithm should be selected according to the **mathematical structure of the problem**.

- DP was selected for project selection because 0/1 Knapsack does not satisfy the greedy-choice property.
- Greedy was selected for job scheduling because selecting the highest-profit job and placing it in the latest available slot gives an optimal solution for this problem.
- A major challenge was building and backtracking the DP table correctly.
- For Greedy scheduling, correctly handling the latest valid slot was important.
- A possible improvement is a **hybrid approach**, using Greedy for fast approximate solutions and DP when the capacity is small enough.
- The project demonstrates the difference between **optimality and efficiency** in algorithm design.

8. DELIVERABLES CHECKLIST

- Pseudocode — Section 2.
- Implementation & Results — Sections 3 and 4 (source code + program output).
- GitHub Upload — push `smart_resource_allocation.c`, `performance_benchmark.c`, and this report to a GitHub repository and share the repository link with the submission.